



Operating Systems & Networks

CTEC1704C

Lecture - 5 : CPU Architecture

Dr. Sameer Jha, SFHEA
Faculty of Computing
De Montfort University Cambodia



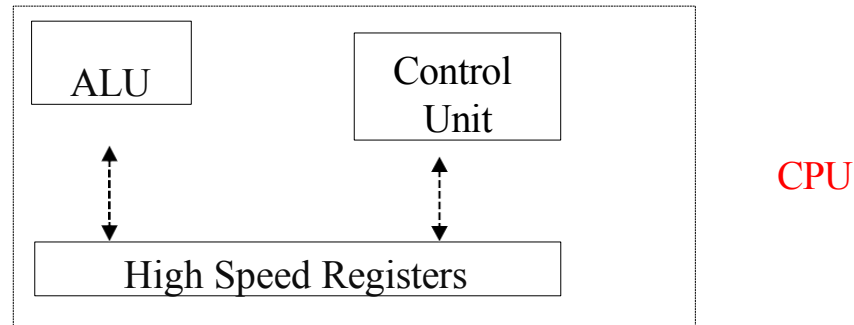
Outline

- CPU Architecture and Function
- What is a Register?
- CPU functions
- CPU performance
- Pipelining



Central Processing Unit Architecture

- The Central Processing Unit is composed of three main components:
 - **Control Unit** - to fetch & decode instructions
 - **ALU** to perform arithmetic($*/+-$) and logical operations (AND,OR,NOT)
 - **Registers** to store temporary data.



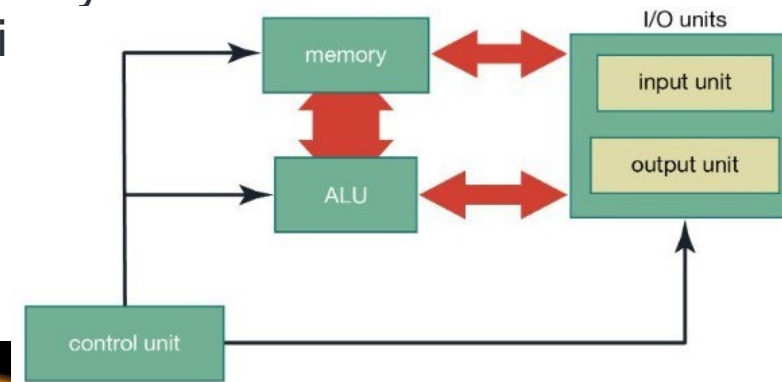
The number and design of registers are also crucial to CPU architecture



Common CPU components and their function

Control unit

- Controlling of data and transfer of data and instructions is done by the control unit among other parts of the computer.
- The control unit is responsible for managing all the units of the computer.
- The main task of the [control unit](#) is to obtain the instructions or data that is input from the memory unit, interpret them, and then direct the operation of the computer according to that.
- The control unit is responsible for communication with Input and output devices for the transfer of data or results from memory.
- The control unit is responsible for managing of data or storing data.





Control Unit

The control unit decides *what happens next* in the CPU.

Example to mention

- Buffer overflow attacks redirect the control unit to malicious instructions

Endpoint security relevance

- Malware alters execution flow using control logic
- Exploits hijack control flow (e.g. return-oriented programming)
- Control-flow integrity (CFI) is a modern defence



ALU (Arithmetic Logic Unit)

ALU (Arithmetic Logic Unit) is responsible for performing arithmetic and logical functions or operations. It consists of two subsections, which are:

- Arithmetic Section: By arithmetic operations, we mean operations like addition, subtraction, multiplication, and division, and all these operations and functions are performed by ALU.
- Logic Section: By Logical operations, we mean operations or functions like selecting, comparing, matching, and merging the data, and all these are performed by ALU.

Note: The CPU may contain more than one ALU and it can be used for maintaining timers that help run the computer system.



Conditional Operations

- Branch to a labeled instruction if a condition is true
 - Otherwise, continue sequentially
- `beq rs, rt, L1` (branch if equal)
 - if (`rs == rt`) branch to instruction labeled L1
- `bne rs, rt, L1` (branch if not equal)
 - if (`rs != rt`) branch to instruction labeled L1
- `j L1` (jump!)
 - unconditional jump to instruction labeled L1



Compiling If Statements

- C code:

```
if (i==j)
    f = g+h;
else
    f = g-h;
```

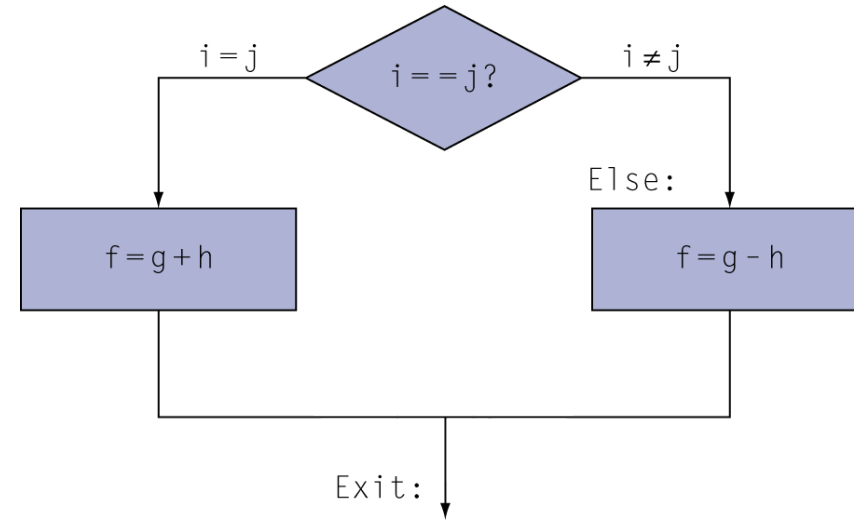
- f, g, ... in \$s0, \$s1, ...

- Compiled MIPS code:

```
bne $s3, $s4, Else
add $s0, $s1, $s2
j    Exit
```

```
    Else: sub $s0, $s1, $s2 # f=g-h (skip if i==j)
```

```
Exit: ...
```



go to 'Else' if i!=j
f=g+h (skip if i!=j)
go to 'Exit'

Assembler calculates addresses



Shift Operations

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- shamt: how many positions to shift
- Shift left logical
 - Shift left and fill with 0 bits
 - sll by i bits **multiplies** by 2^i
- Shift right logical
 - Shift right and fill with 0 bits
 - srl by i bits **divides** by 2^i (unsigned only)



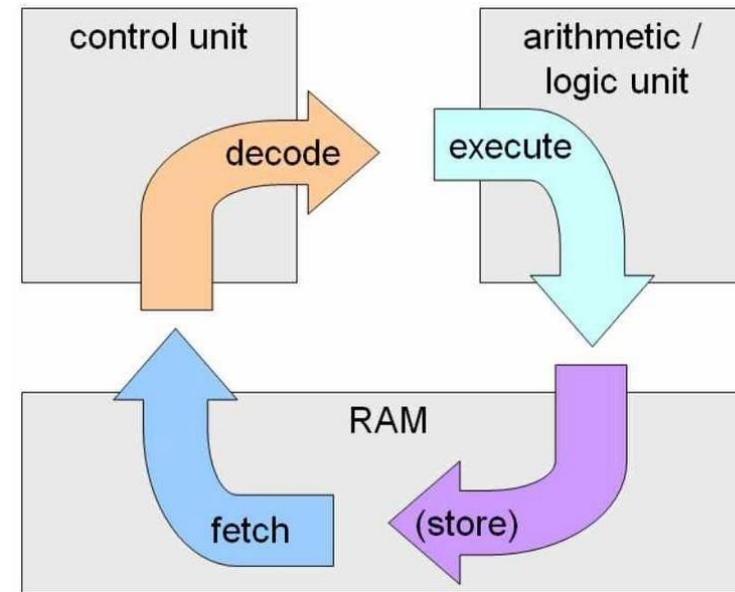
CPU Function

What Does a CPU Do?

The main function of a computer processor is to execute instructions and produce an output.

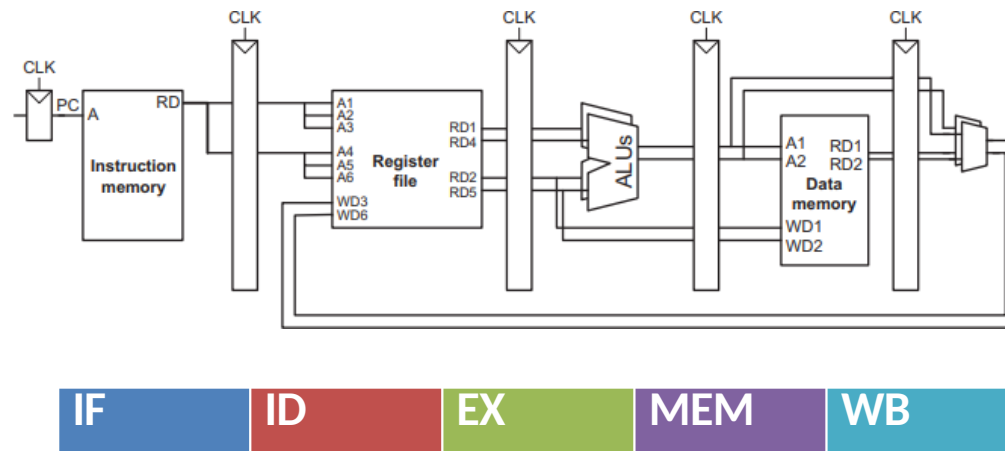
Fetch, **Decode**, and **Execute** are the fundamental functions of the computer.

- **Fetch**: the first CPU gets the instruction. That means binary numbers that are passed from RAM to CPU.
- **Decode**: When the instruction is entered into the CPU, it needs to decode the instructions. with the help of ALU(Arithmetic Logic Unit), the process of decoding begins.
- **Store**: After the execute step the instructions are ready to store in the memory to execute



CPU Instruction Cycle

- Programs consist of many instructions
- Processor follows an **instruction cycle** to execute each instruction
- Which steps do we need in an instruction cycle?
 - IF: Instruction fetch from memory
 - ID: Instruction decode & register read
 - EX: Execute operation or calculate address
 - MEM: Access memory operand
 - WB: Write result back to register



This is a core security concept :
Malware and security software both rely on the same instruction cycle.

IF

- Load PC into instruction memory, fetch instruction

ID

- Look up register numbers in register file, read registers

EX

- **Depending on instruction class**

- Use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch target address
- Access data memory for load/store
- For next cycle: $PC = \text{target address or } PC + 4$

MEM



CPU performance

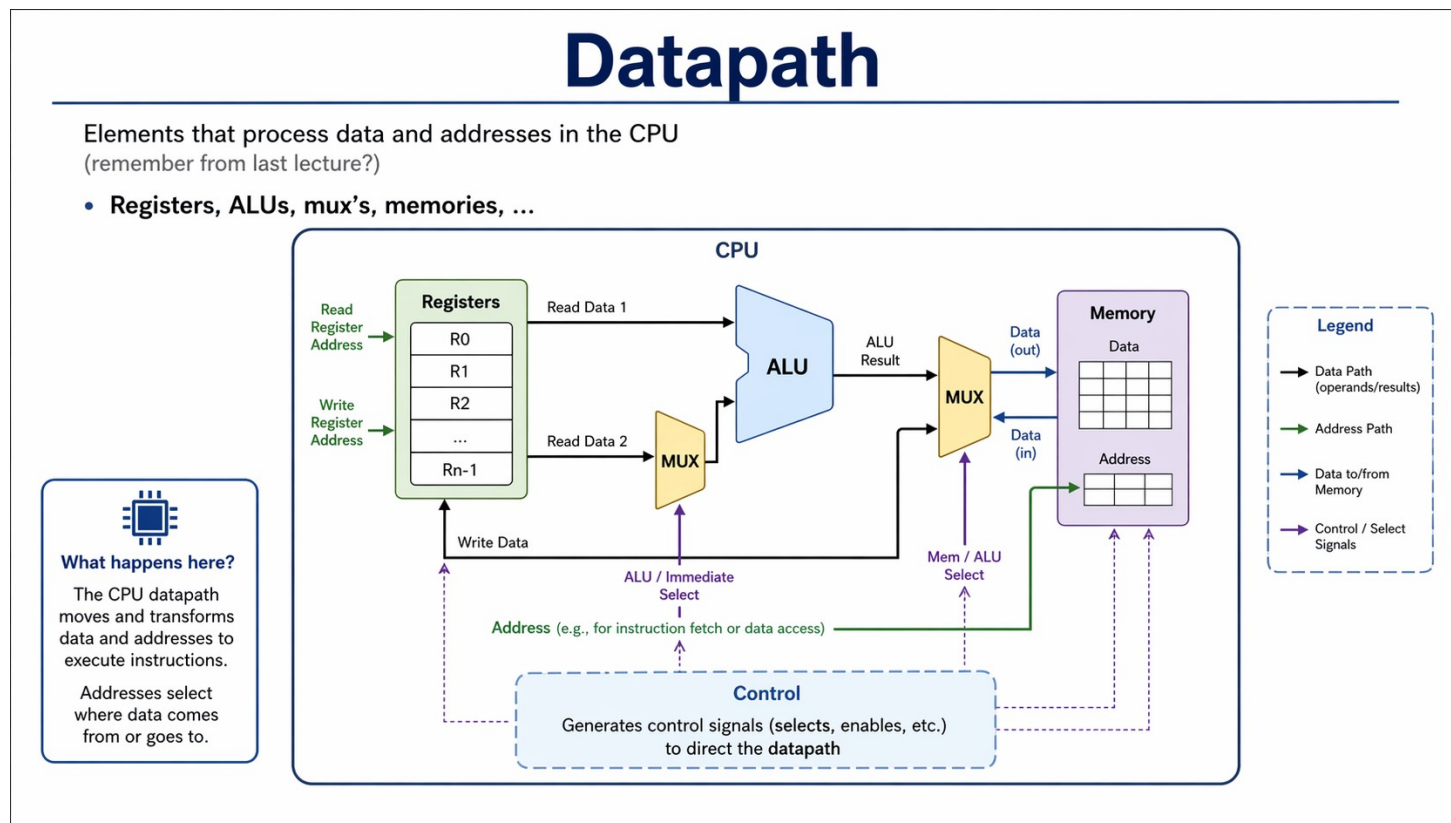
Even though today's processors are tremendously fast, their performance can be affected by a number of factors:

- Clock speed
- Cache memory size
- Number of cores

Real-world example

- Cryptomining malware pushes CPU clock usage to maximum
- Endpoint tools flag **sustained high CPU usage**

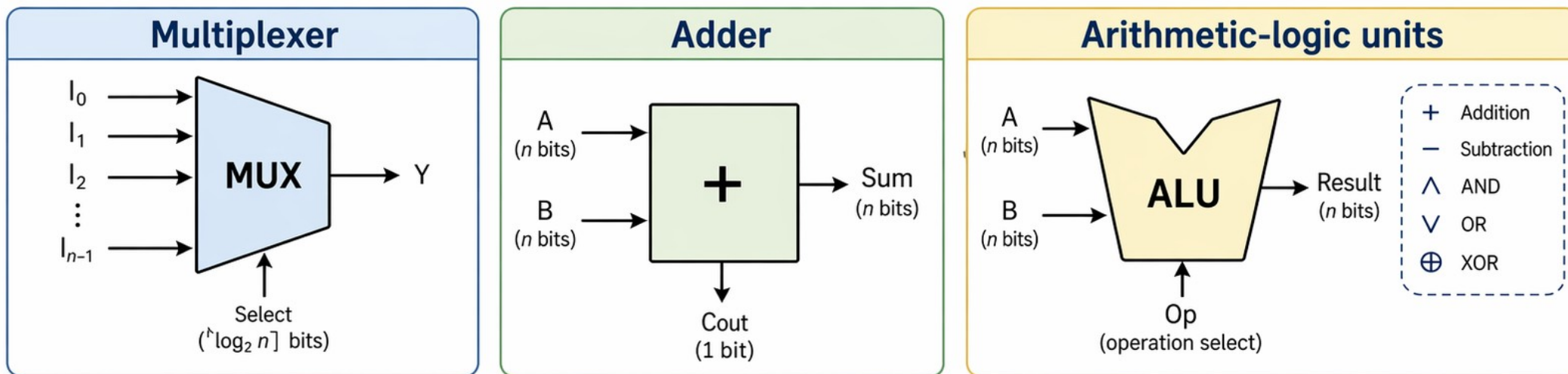
- Datapath
 - Elements that process data and addresses in the CPU
 - Registers, ALUs, mux's, memories, ...





Processor runs instructions

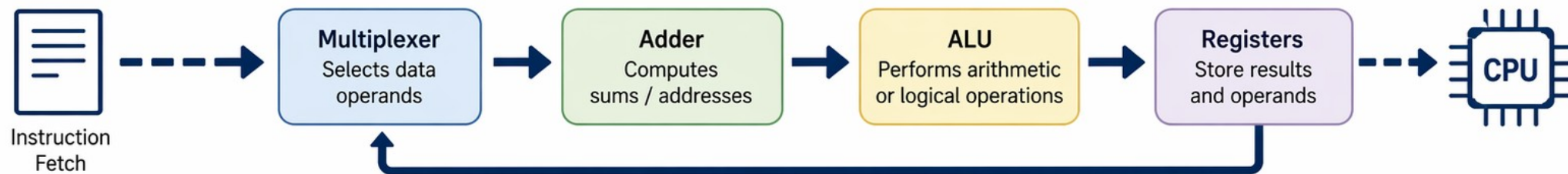
These three examples of combinational circuits are important elements of processor hardware:



Selects one input from multiple sources.

Performs binary addition.

Executes arithmetic and logical operations.

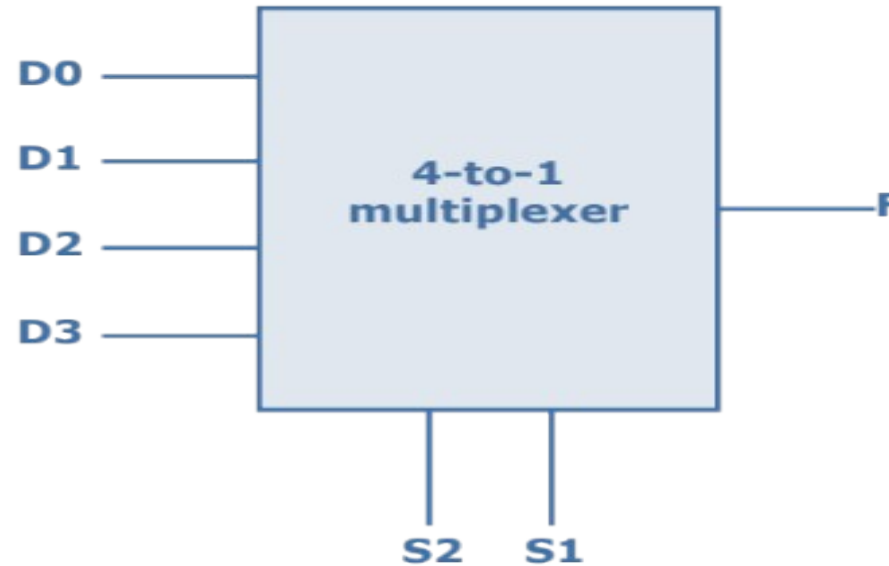


Data flows through combinational circuits to execute each instruction



Multiplexer

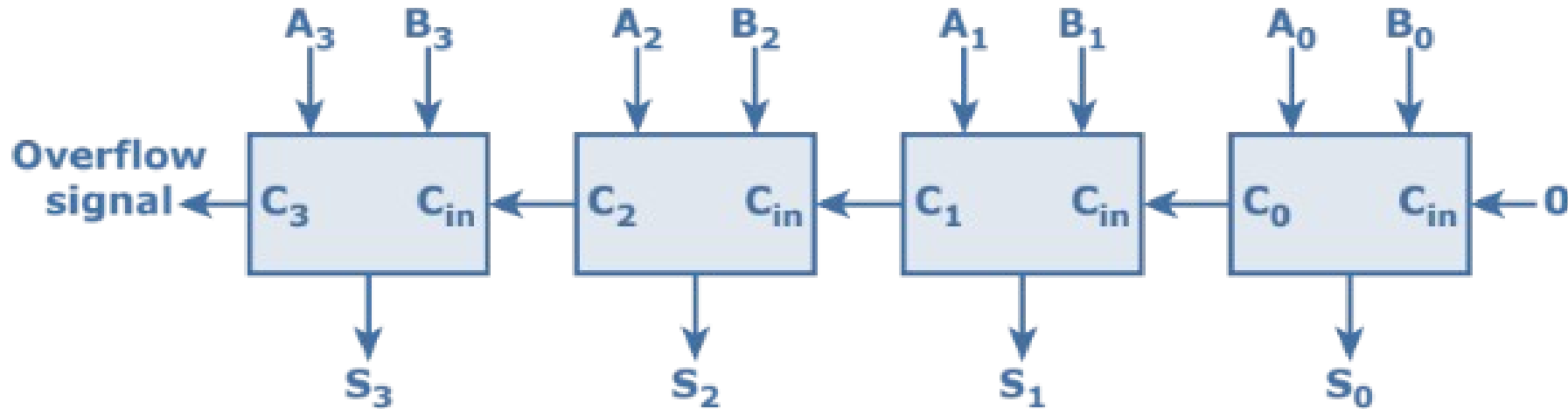
- Used to select one from several inputs (D)
- Selection code (S) specifies which input





4-bit Adder

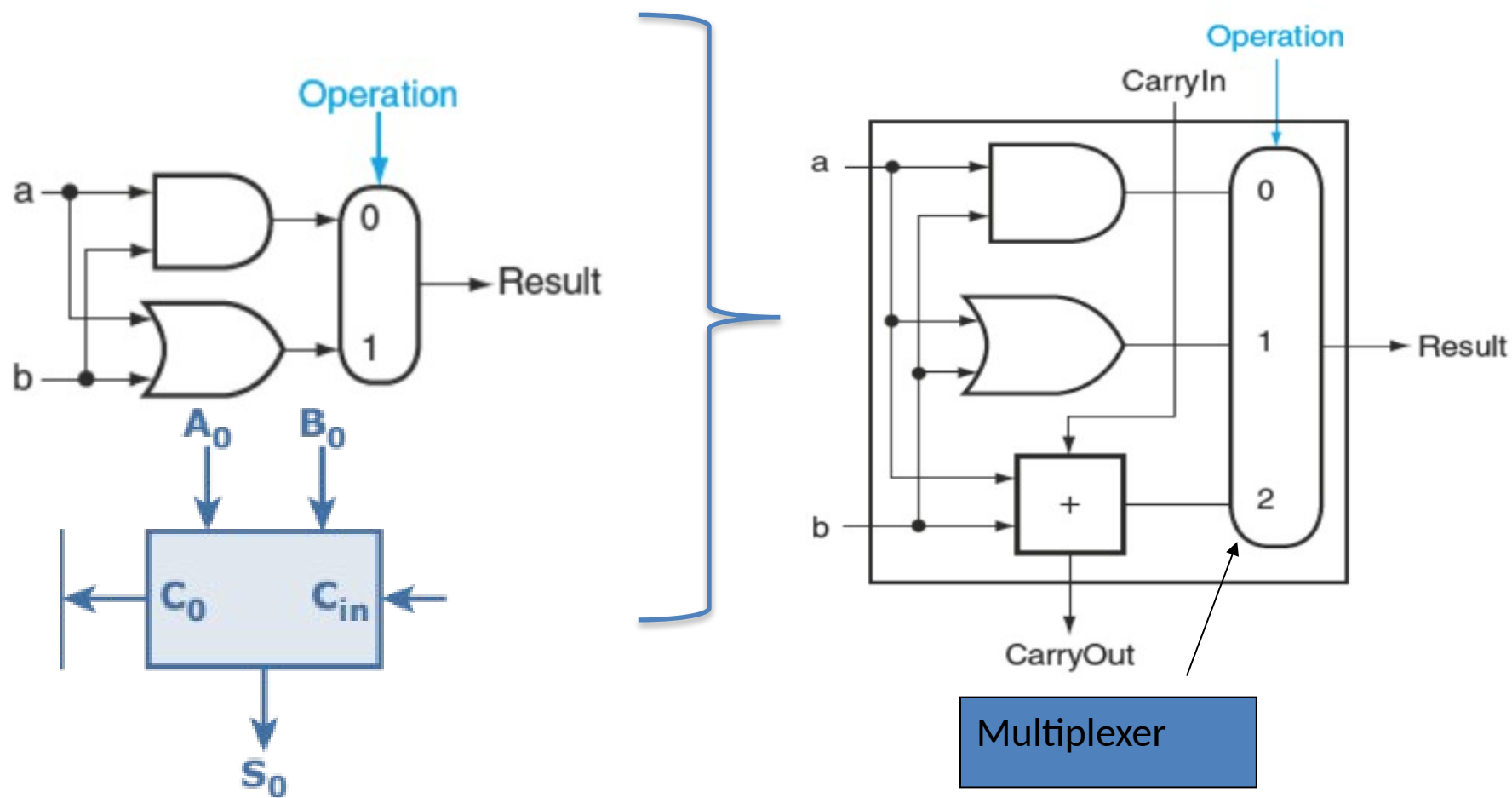
We can combine single-bit adders by chaining the carry bits: connect carry-out of adder 0 to carry-in of adder 1





1-bit Arithmetic-Logic Unit (ALU)

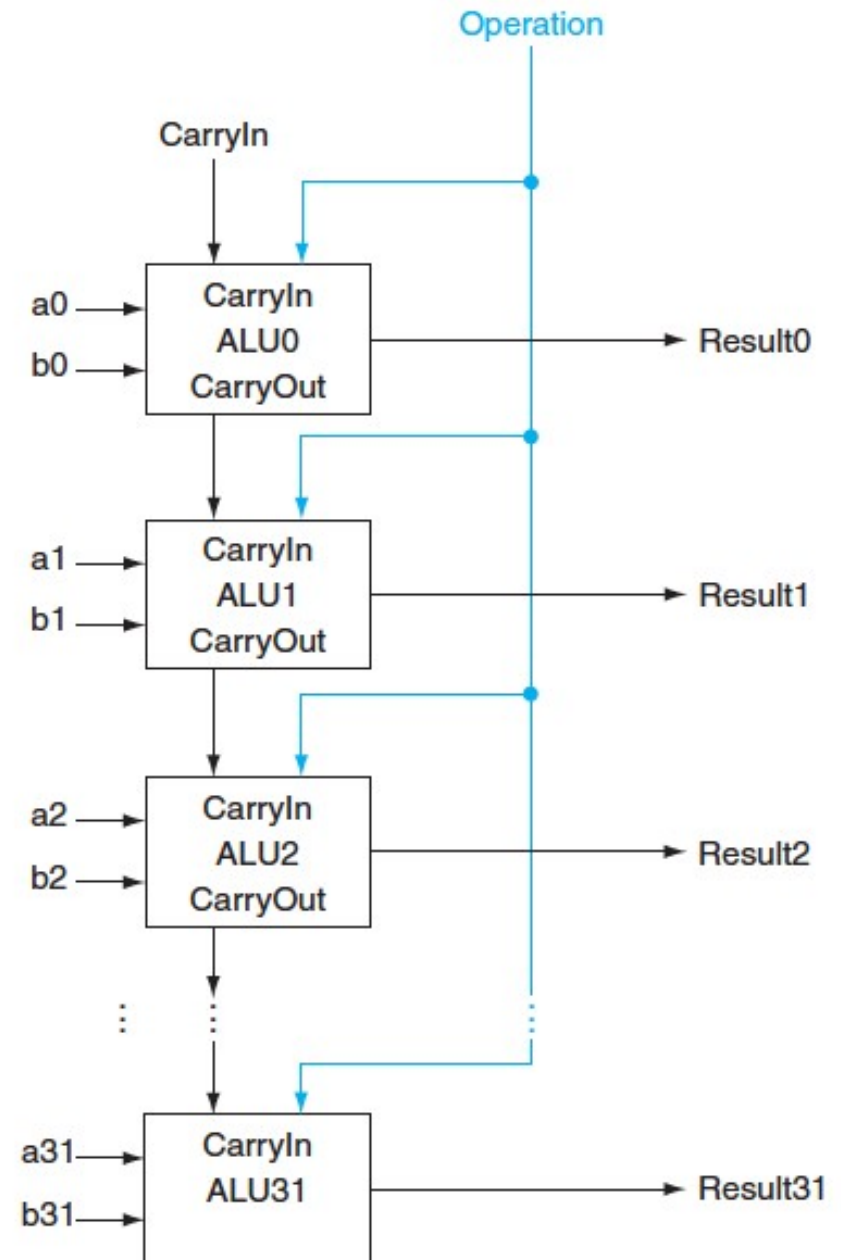
Combine 1-bit AND/OR unit with 1-bit adder





32-bit ALU

- Chain 32 1-bit ALUs using the carry bits
- The result bits from the 32 1-bit ALUs form the 32-bit result

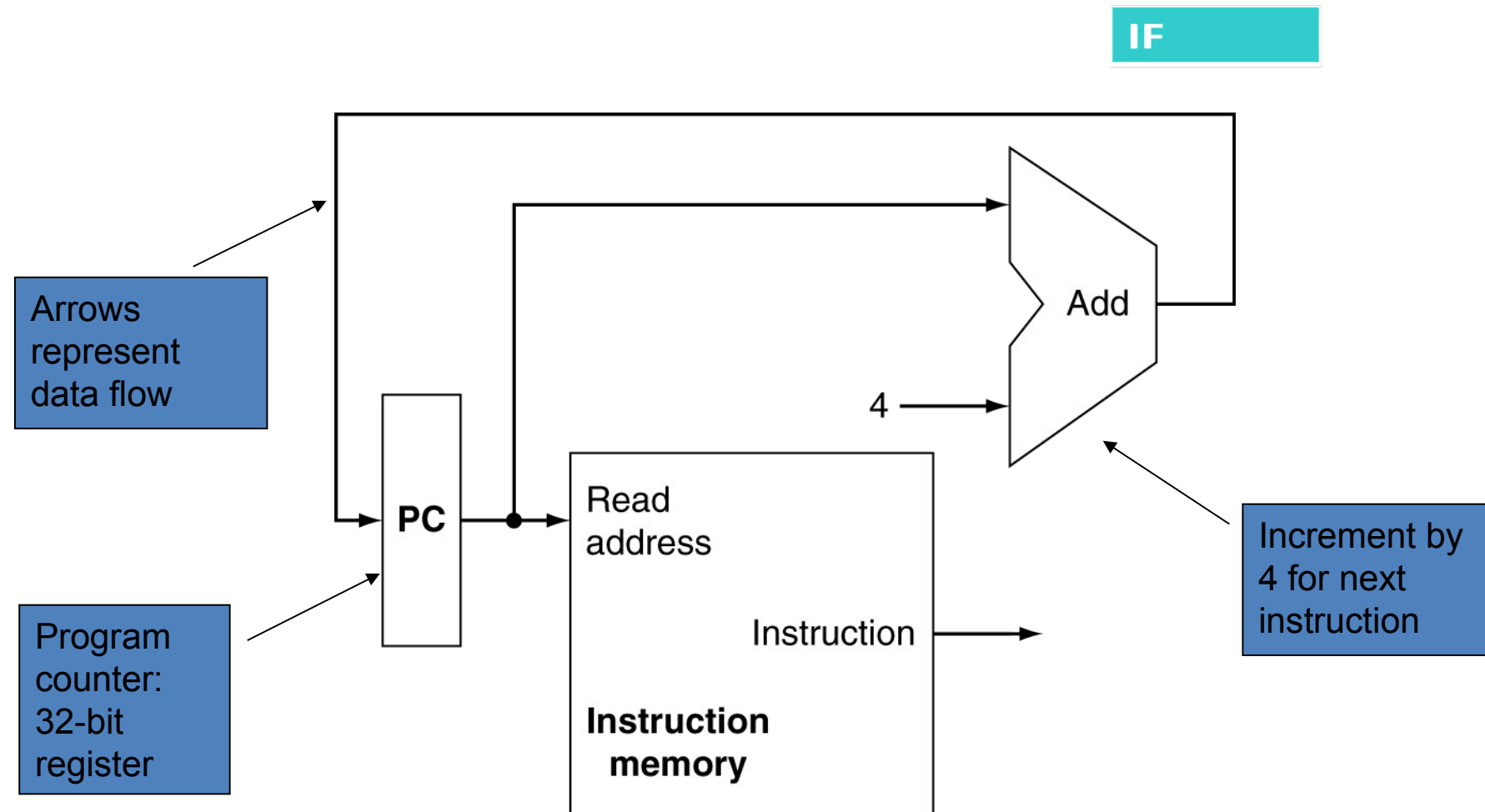




How do they come together in a DataPath??



Start with Instruction Fetch





Arithmetic/Logical Instructions

ID

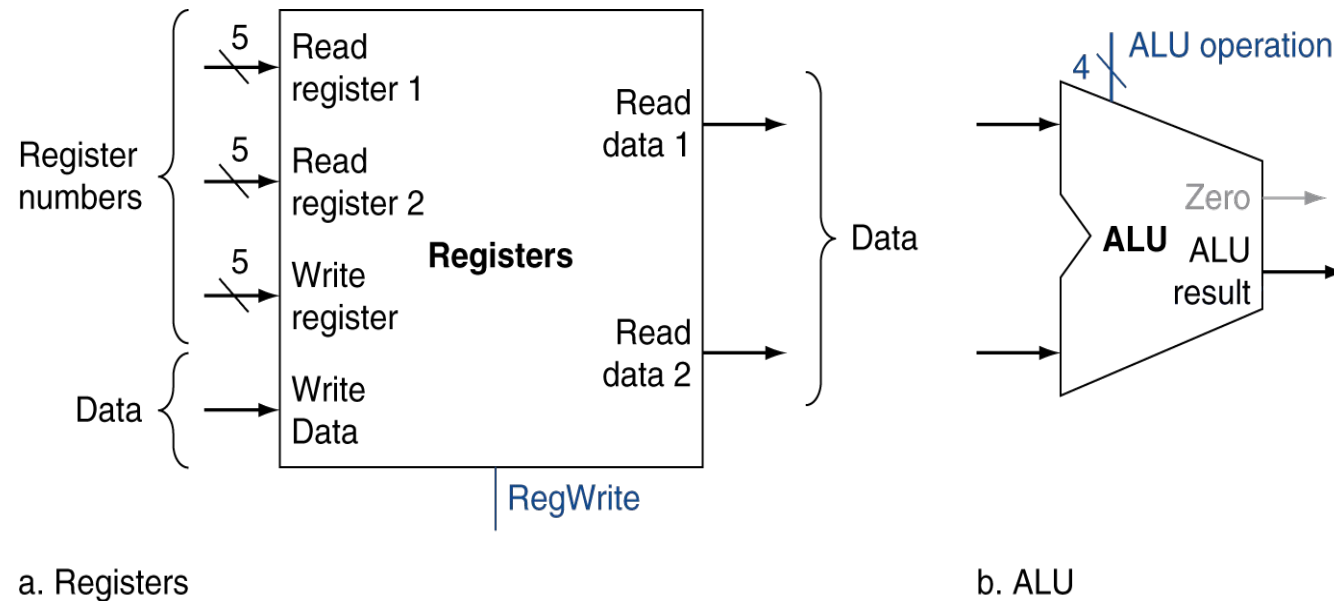
- Read two register operands

EX

- Perform arithmetic/logical operation

WB

- Write register result





Load/Store Instructions

ID

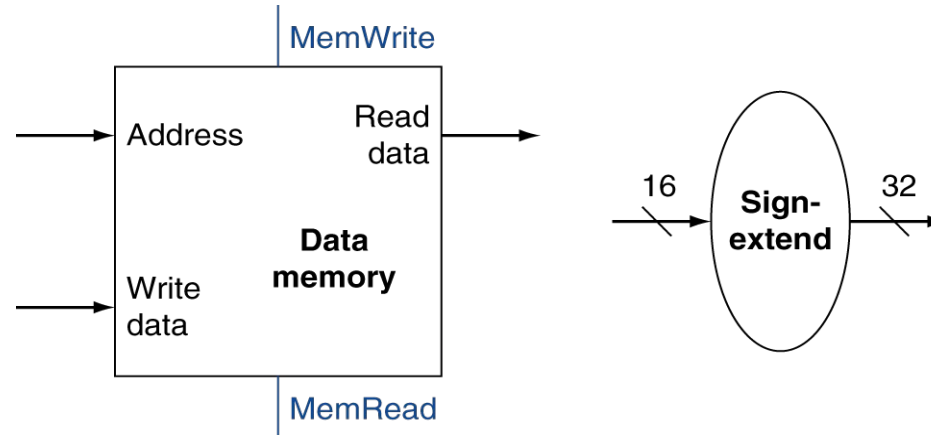
- Read register operands

EX

- Calculate address using 16-bit address
 - Use ALU, but sign-extend address (needed because instruction supplies 16 bits, but ALU expects 32)

MEM

- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit

b. Sign extension unit

ID

- Read register operands

EX

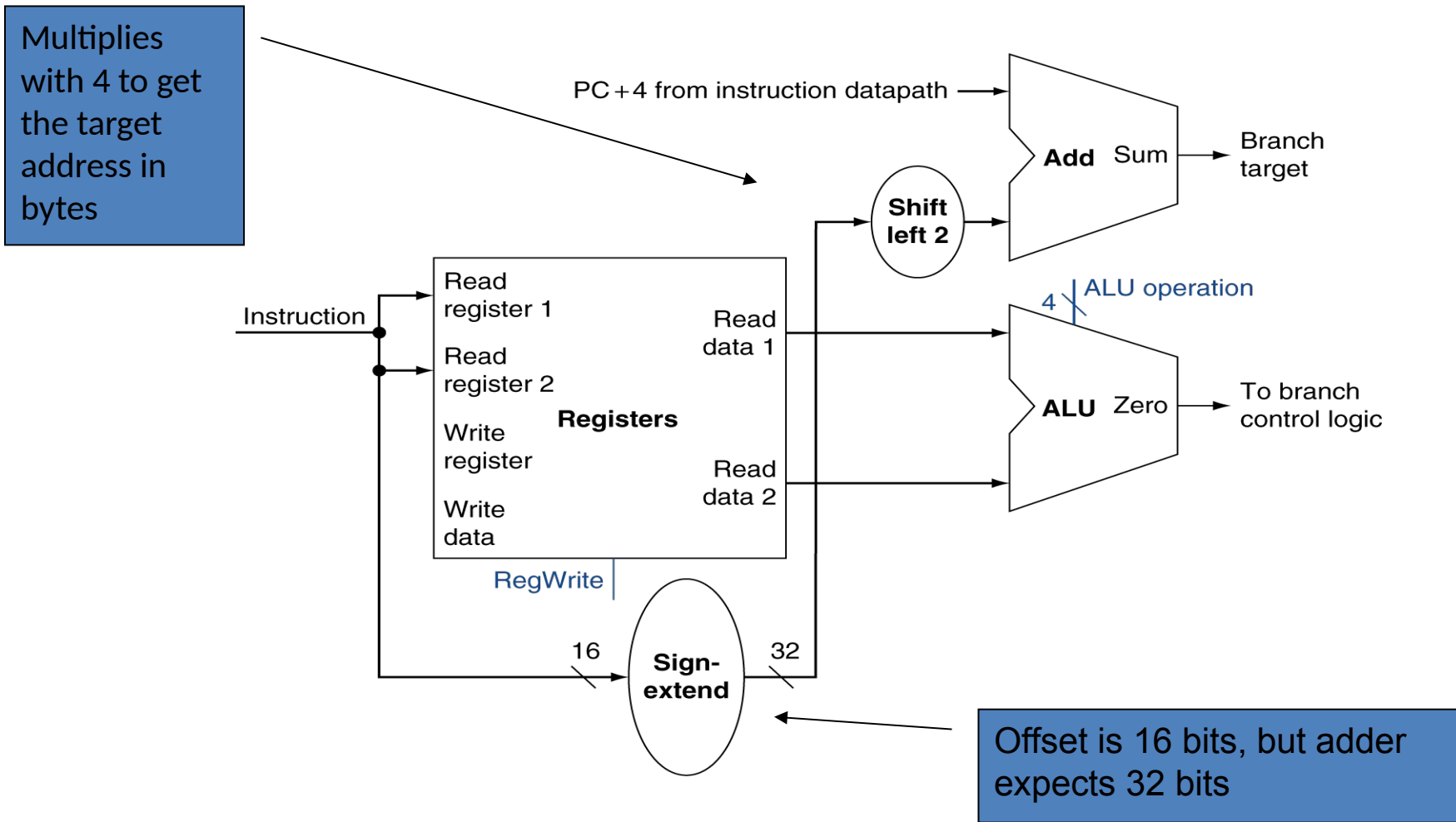
- Compare operands

- Use ALU, subtract and check Zero output

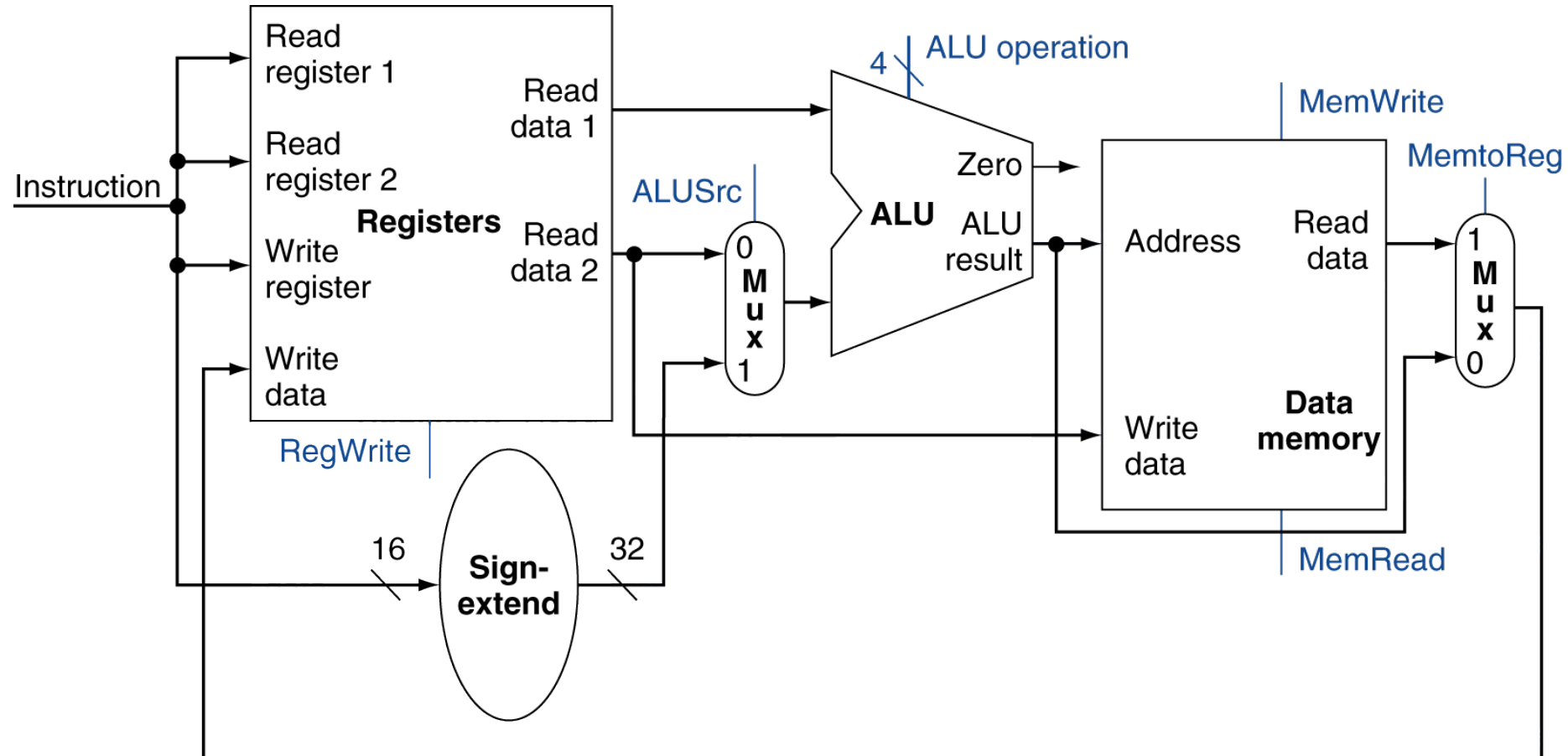
EX

- Calculate target address

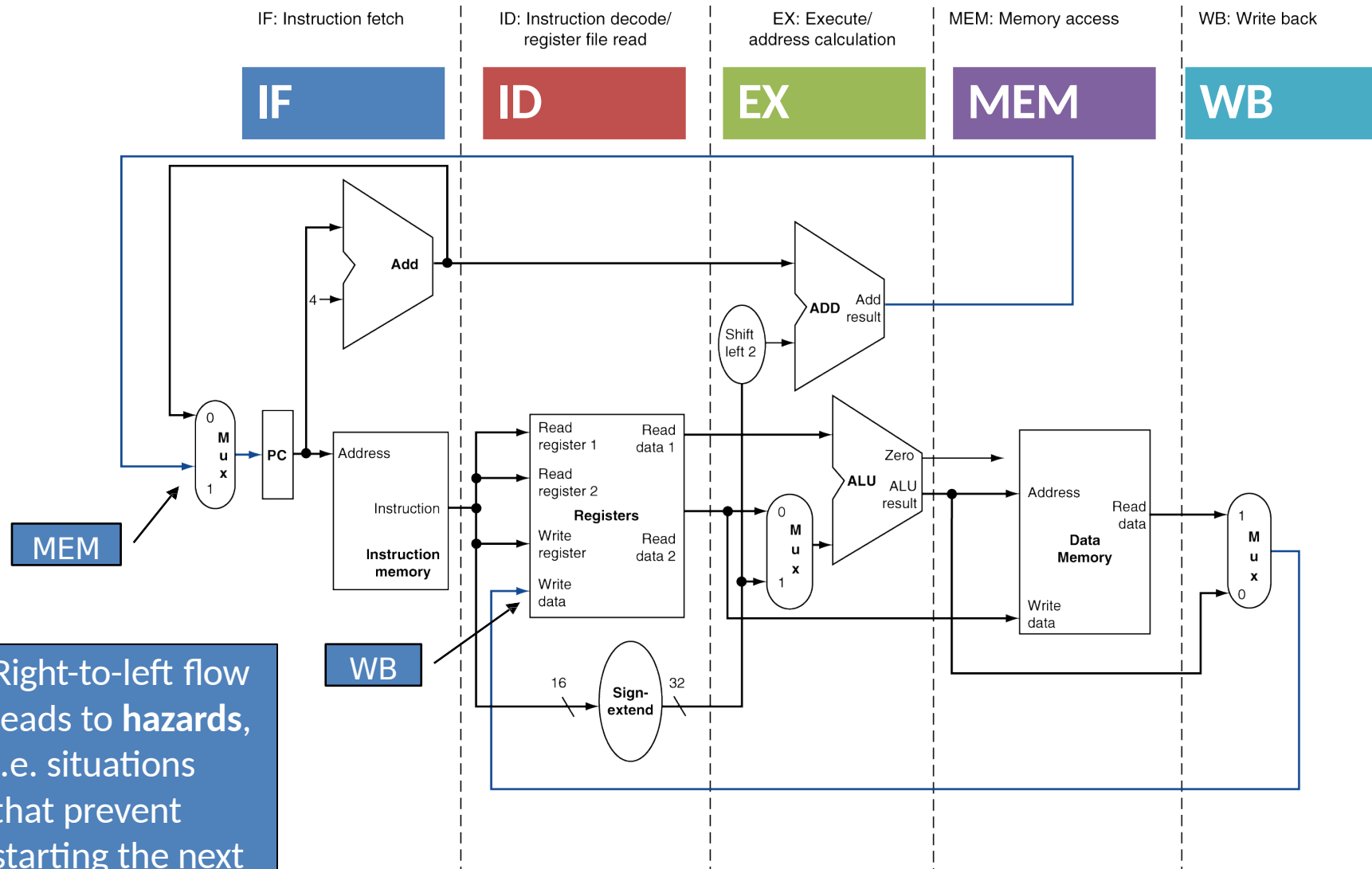
- Sign-extend 16-bit address (label)
- Shift left 2 places (address is given in **words**, i.e. multiples of 4 bytes. We shift left to multiply with 4 and get the target address in bytes)
- Add to PC + 4
 - This is done by the instruction fetch stage



Arithmetic/Logical/Load/Store Datapath

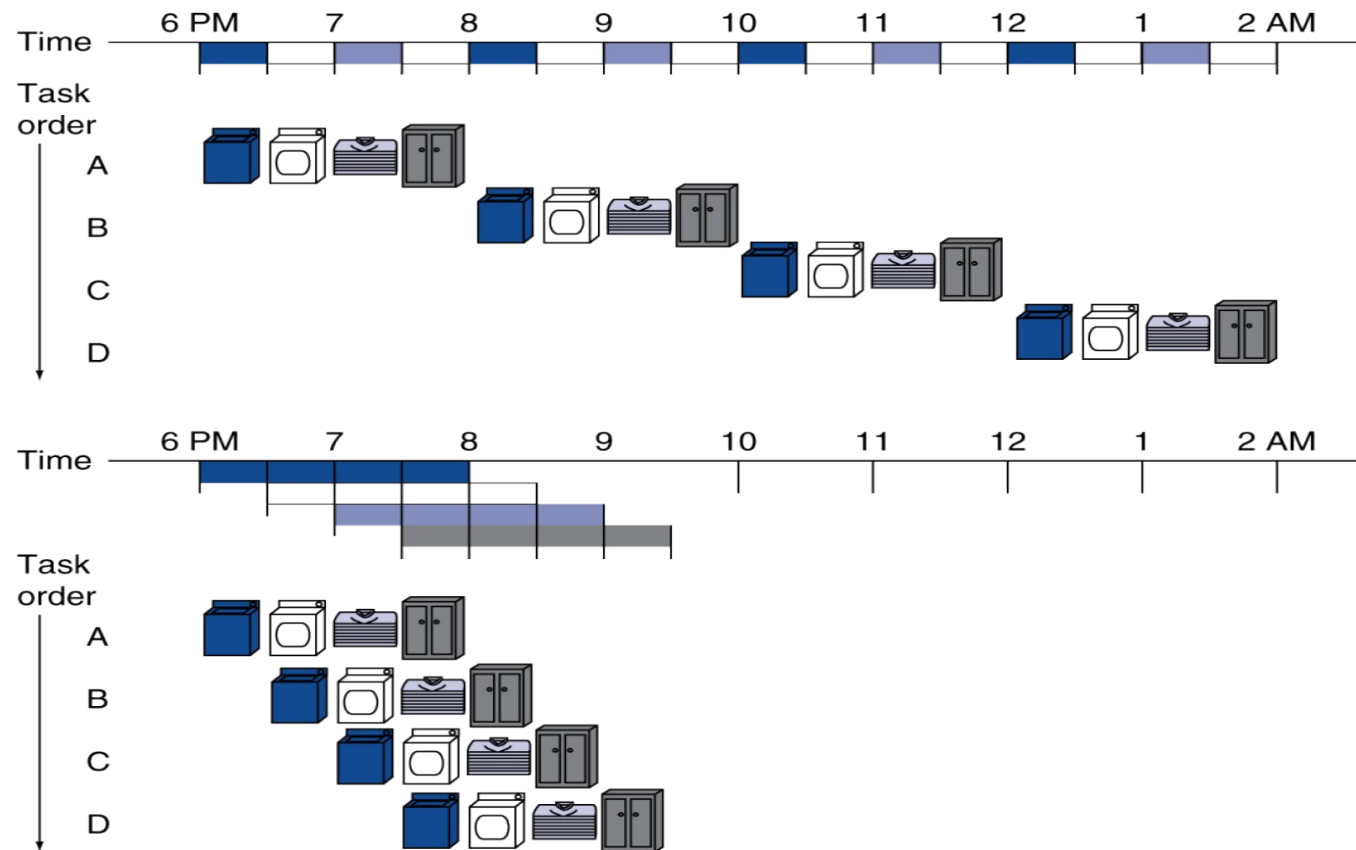


MIPS Pipelined Datapath



Pipelining Analogy

- Pipelined laundry: overlapping execution
 - Parallelism improves performance





- You see, everything is MATHS.
- At the EOD, Mathematical operations and logical diagrams model the world.
- These models are run on computers with the help of hardware components, building up on layers and layers of abstractions .
- But at the very base of it, it's basic maths and logic, and we learnt how it works.



Thank You!